Cupcake - 17-04-2026

Navn	cph-email	Hold	Github-navm
Jasmin Agar	cph-ja482@stud.ek.dk	Hold b	jasminagar
Melissa Bekirova	cph-mb1122@stud.ek.dk	Hold b	Melissabekirova04

Link til video:

https://youtu.be/EbJsugGml2U

Indholdsfortegnelse

Cupcake - 17-04-2026 **1**
Indholdsfortegnelse **2**
Indledning **3**
Baggrund **3**
Teknologi valg **3**
Krav **3**
Er-diagram **4**
Klassediagram **5**
Domæne model (Bilag 3) **5**
Figma (bilag 5,6 og 7): 5
Særlige forhold **6**
Status på implementation **7**
Proces **7**
Bilag **8**

Indledning

Denne rapport omhandler udviklingen af et webbaseret system til håndtering af bestilling af cupcakes. Systemet gør det muligt for brugere at oprette en konto, logge ind, sammensætte cupcakes ved at vælge bund og topping, lægge dem i en kurv og gennemføre en ordre. Derudover er der en admin-funktionalitet, hvor man kan se alle ordrer og administrere brugere.

Rapporten er skrevet til en fagfælle, som er en datamatikerstuderende på 2. semester, men ikke nødvendigvis kender cupcake-opgaven på forhånd. Derfor vil centrale dele af systemet og designvalg blive forklaret, så det er muligt at følge med i både struktur og implementering.

Formålet med projektet har været at arbejde med samspillet mellem backend, database og webapplikation, herunder brug af MVC-struktur, databasedesign og håndtering af brugerinput. Samtidig har der været fokus på at lave en løsning, der er logisk opbygget og realistisk i forhold til, hvordan et simpelt bestillingssystem kunne fungere i praksis.

Baggrund

Vi har udarbejdet denne applikation for virksomheden **Olskers cupcakes**. Virksomheden sælger cupcakes, som sammensættes af en bund og en topping. **Olskers cupcakes** er ejet af iværksættere fra Bornholm, som med en masse kreativitet har formået at sammensætte den helt perfekte opskrift. Derudover er deres cupcakes lavet på økologiske råvarer, hvilket også bidrager til den gode smag.

Teknologi valg

Projektet er udviklet i Java 21 og bygger med Apache Maven (maven-compiler-plugin 3.10.1 og maven-shade-plugin 3.4.1). Udviklingen er udført i IntelliJ IDEA 2023.3.5.

Backend-delen er implementeret med Javalin 6.7.0 samt Javalin-rendering 6.5.0, som håndterer webserver og routing. Til præsentationslaget anvendes Thymeleaf 3.1.3.RELEASE sammen med Thymeleaf-extras-java8time 3.0.4.RELEASE til håndtering af Java 8+ tidsobjekter i views.

Datahåndtering sker via PostgreSQL, hvor forbindelsen etableres gennem PostgreSQL JDBC-driver version 42.7.5, og connection pooling varetages af HikariCP 6.2.1 for effektiv ressourcehåndtering.

Krav

Firmaets håb

Formålet med systemet er at digitalisere og effektivisere virksomhedens bestillingsproces. Ved at tilbyde en webbaseret løsning, hvor kunder selv kan sammensætte cupcakes med valgfri bund og topping, understøtter systemet en fleksibel og brugerdrevet

bestillingsoplevelse. Systemet tilfører værdi ved at muliggøre online bestilling og betaling, hvilket reducerer manuel håndtering og minimerer risikoen for fejl i ordrer. Samtidig giver det virksomheden et bedre overblik over indkomne bestillinger og planlægning af produktion i forhold til afhentningstidspunkter.

Derudover bidrager løsningen til øget effektivitet i driften samt forbedret kundeservice, da kunderne oplever en mere smidig proces, hvor de blot skal afhente deres ordre. På længere sigt understøtter systemet også virksomhedens digitale tilstedeværelse og skalerbarhed i forhold til vækst.

User-stories

US-1: Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken i Olsker og hente min ordre.

US-2 Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en en ordre.

US-3: Som administrator kan jeg indsætte beløb på en kundes konto direkte i Postgres, så en kunde kan betale for sine ordrer.

US-4: Som kunde kan jeg se mine valgte ordrelinier i en indkøbskurv, så jeg kan se den samlede pris.

US-5: Som kunde eller administrator kan jeg logge på systemet med email og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side (evt. i topmenuen, som vist på mockup'en).

US-6: Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.

Er-diagram

Bilag 1 viser, hvordan databasen er bygget op omkring et klassisk order flow, hvor en bruger kan have flere ordrer, og hver ordre kan indeholde flere order_lines. Det giver en fleksibel måde at håndtere bestillinger på, især når en ordre består af flere forskellige produkter.

Kernen i designet er order_lines-tabellen, som fungerer som en kobling mellem orders, bottoms og toppings. Hver linje repræsenterer en specifik kombination af bund, topping og antal, hvilket gør det muligt at samle flere forskellige cupcakes i én ordre.

I designet har vi haft fokus på normalisering, så data er opdelt i flere tabeller for at undgå redundans og problemer ved opdateringer. For eksempel er bottoms og toppings gemt hver for sig, så ændringer kun skal laves ét sted. Samtidig har vi forsøgt at holde strukturen enkel og overskuelig, så den er nem at arbejde med i koden og videreudvikle uden at skabe unødvendig kompleksitet.

Vi har også valgt at gemme både line_price og total_price. Det er gjort for at sikre, at gamle ordrer ikke ændrer sig, hvis priserne senere bliver opdateret. På den måde bevarer vi en korrekt historik i systemet.

Klassediagram

Vi udarbejdede klassediagrammet (bilag 2) som noget af det første, inden vi begyndte at kode, og det er derfor mindre og simplere end det færdige system. Vi havde en lagdelt opdeling i tankerne gennem hele udviklingsprocessen, som klassediagrammet skulle afspejle. Formålet var at skabe en klar struktur mellem controllers, services og datalaget, så udviklingen blev mere overskueligt. Særligt service-laget blev justeret undervejs, da den oprindelige opdeling viste sig at samle for meget ansvar ét sted.

Samlet set fungerede klassediagrammet som et godt udgangspunkt, men vi måtte løbende tilpasse vores system som det voksede.

Domæne model (Bilag 3)

Domænemodellen er udarbejdet tidligt i udviklingsprocessen for at skabe overblik over de centrale begreber i systemet og deres relationer. Fokus har været på at modellere problemområdet, herunder brugere, ordrer og cupcakes, frem for tekniske detaljer. Modellen blev brugt som grundlag for den videre udvikling og blev løbende justeret i takt med en bedre forståelse af domænet.

Figma (bilag 5,6 og 7):

Da vi ville have mere fokus på designet under dette projekt, brugte vi Figma som værktøj til at designe og planlægge vores hjemmeside. Vores fokus var at hjemmesiden skulle fremstå indbydende, og passe temaet om cupcakes. Vi gjorde dette ved hjælp af farver, billeder, og typografi. Vi designede først forsiden med ovenstående i fokus, og fik derefter til design af kurven. Da dette skulle være en popup, måtte vi lave en kopi af hjemmesiden, og indsætte kurven på den. Vi lavede en underside med cupcakes, hvor brugeren kan se udvalg, priser og muligheder for at sammensætte sine egne cupcakes. Dette gav os et overblik over brugerens vej gennem hjemmesiden og hvordan man navigerer mellem siderne.

Da vi har haft fokus på, at designet skulle passe til temaet, valgte vi en farvepalette med lilla og lyserøde nuancer, som giver et sødt og indbydende udtryk, der matcher produktet. Vi har også brugt billeder af cupcakes og en blød skrifttype for at understøtte stemningen.

Samlet set har Figma hjulpet os med at visualisere vores idéer, strukturere indholdet og skabe en rød tråd i designet, før vi begyndte at kode hjemmesiden.

Særlige forhold

Before-filter

```
app.before(ctx -> {
   User user = ctx.sessionAttribute("currentUser");
   if (user != null) {
       ctx.attribute("currentUser", user);
   }
});

app.before("/order", ctx -> {
   if (ctx.sessionAttribute("currentUser") == null) {
       ctx.redirect("/login");
   }
});

app.before("/admin", ctx -> {
   User user = ctx.sessionAttribute("currentUser");
   if (user == null || !"admin".equals(user.getRole())) {
       ctx.redirect("/login");
   }
});
```

Vi bruger **before** til at styre både session og adgangskontrol i vores applikation. Først har vi lavet et globalt filter, hvor vi henter **currentUser** fra sessionen. Hvis der ligger en bruger i sessionen, sætter vi den som en attribute, så vi kan tilgå brugeren i hele requestet og i vores views uden at skulle hente den igen hver gang.

Derudover har vi beskyttet bestemte routes. På **/order** tjekker vi, om der findes en `currentUser` i sessionen. Hvis ikke, redirecter vi brugeren til **/login**, så kun loggede brugere kan lave bestillinger. På **/admin** har vi lavet et ekstra sikkerhedslag, hvor vi både tjekker login og rolle. Her kontrollerer vi, om brugeren er logget ind, og om **user.getRole()** er **"admin"**. Hvis en af de betingelser ikke er opfyldt, bliver brugeren sendt tilbage til login-siden. Det betyder, at kun admin-brugere kan tilgå admin-delen af systemet.

Fragments (Bilag 4)

Vi har valgt at lave headeren som et separat Thymeleaf-fragment, så den kan genbruges på tværs af alle sider i systemet. Dette er gjort for at undgå gentagelse af kode og sikre en ensartet brugergrænseflade. Ved at isolere headeren i et fragment kan vi vedligeholde navigation, login-status og kurv ét sted, hvilket gør det lettere at ændre funktionalitet eller design uden at skulle opdatere alle sider individuelt. Det øger både overskueligheden og vedligeholdelsen af koden.

Headeren indeholder blandt andet login-status, log ud/login knapper samt kurv-funktionalitet, som derfor altid er tilgængelig for brugeren uanset side.

Roller i systemet

ALTER TABLE users ADD COLUMN role VARCHAR(20) DEFAULT 'user';

Vi har implementeret rollebaseret adgangskontrol i systemet ved at tilføje en **role**-kolonne i **users**-tabellen. Dette fremgår også af ERD-diagrammet, hvor rollen er en del af bruger-entiteten.

I applikationen anvendes rollen til at styre adgang til forskellige dele af systemet. Dette sker via **before-filters**, hvor vi kontrollerer brugerens rolle, inden der gives adgang til eksempelvis admin-siden. Kun brugere med rollen **"admin"** får adgang, mens andre bliver redirectet til login eller forsiden.

Status på implementation

Vi er nået i mål med de første 6 user stories i forhold til implementationen. Det betyder, at de centrale dele af systemet fungerer, herunder oprettelse af ordrer, håndtering af order lines, login samt grundlæggende funktionalitet for både bruger- og admin-flow.

Efter dette valgte vi at stoppe videre udvikling af de resterende user stories og i stedet fokusere på at tilpasse og færdiggøre de eksisterende dele af systemet. Den resterende tid blev derfor brugt på at sikre, at de grundlæggende funktionaliteter er robuste, og udvikling af rapport og dokumentation.

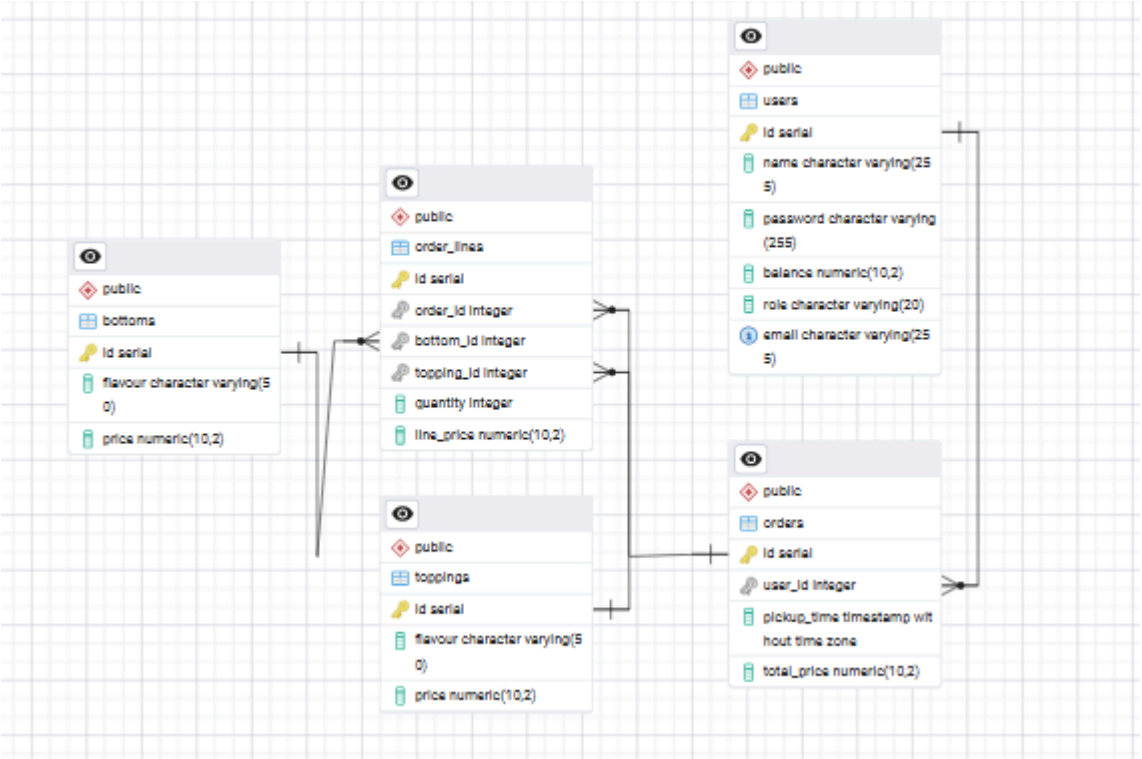
Proces

I starten af projektet lagde vi vægt på at løbende kunne fordele opgaver mellem os og tilpasse planerne undervejs. Da vi er en gruppe på to personer, blev samarbejdet meget direkte, og det gjorde det nemt hurtigt at afklare, hvem der tog hvilke dele af opgaven. Derefter arbejdede vi ofte selvstændigt med hver vores del af systemet, men hjalp stadig hinanden, når der opstod problemer eller behov for sparring. Vi valgte fra begyndelsen at bruge god tid på design. Dette gav os et overblik over systemet, hvilket gav mere ro i sindet under implementationen, og gav os en retning at arbejde ud fra. Vi startede med at implementere user-stories fra toppen og ned, hvilket gjorde at arbejdsprocessen ikke var lige så flydende som vi havde håbet. Vi måtte ofte ind og ændre i, og tilpasse vores løsning, hvilket tog meget af vores tid.

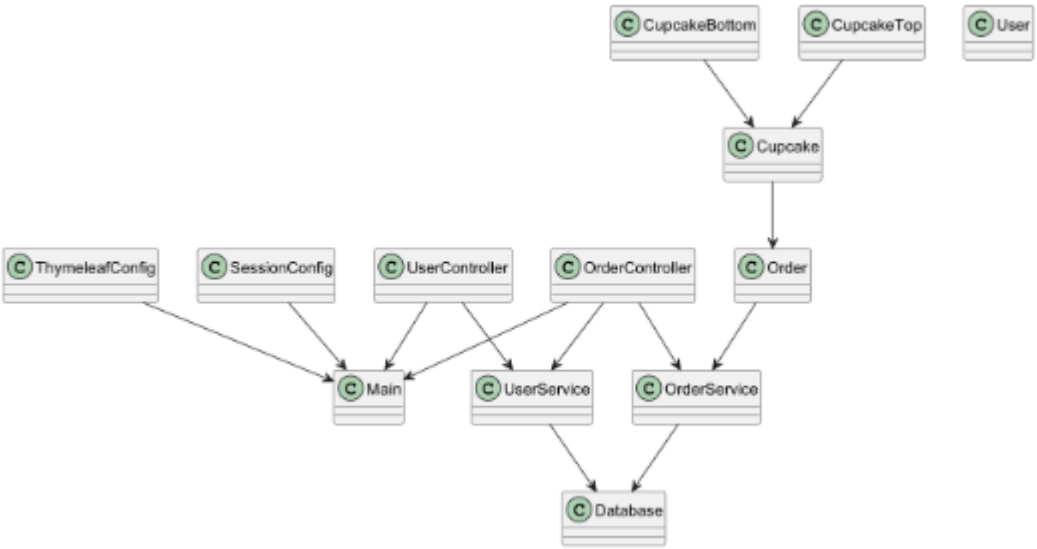
Det der fungerede godt i forløbet var især vores samarbejde og den løbende kommunikation. Det gjorde det nemt at tage beslutninger hurtigt og komme videre uden at sidde fast for længe i diskussioner. Vi fik også testet vores funktioner løbende, hvilket gjorde det lettere at opdage fejl tidligt og rette dem undervejs. Samlet set gav den måde vi arbejdede på et godt flow i projektet, selvom vi undervejs måtte tilpasse nogle dele af vores oprindelige plan.

Bilag

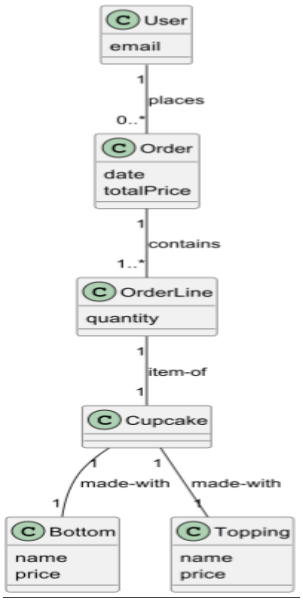
Bilag 1



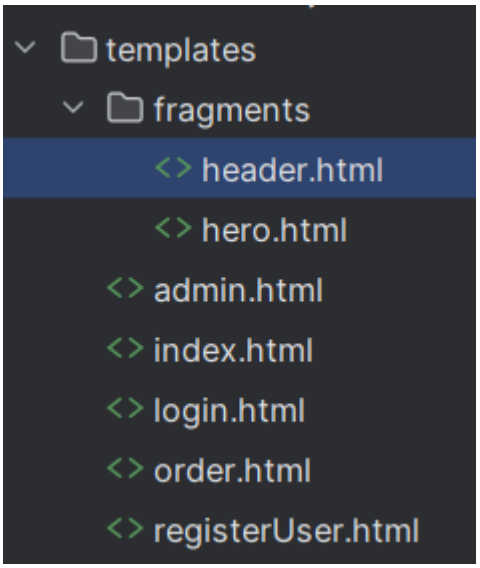
Bilag 2

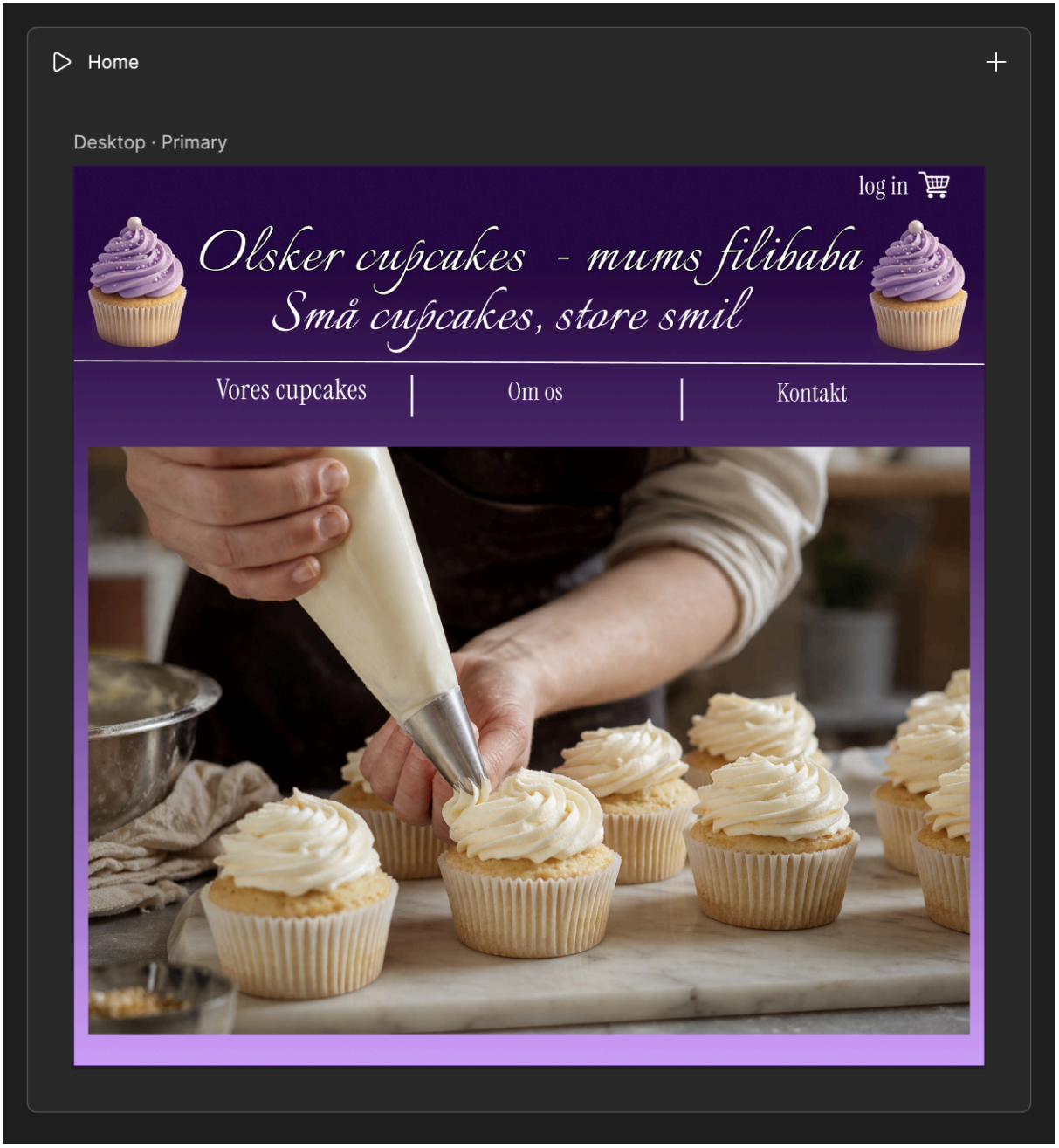


Bilag 3

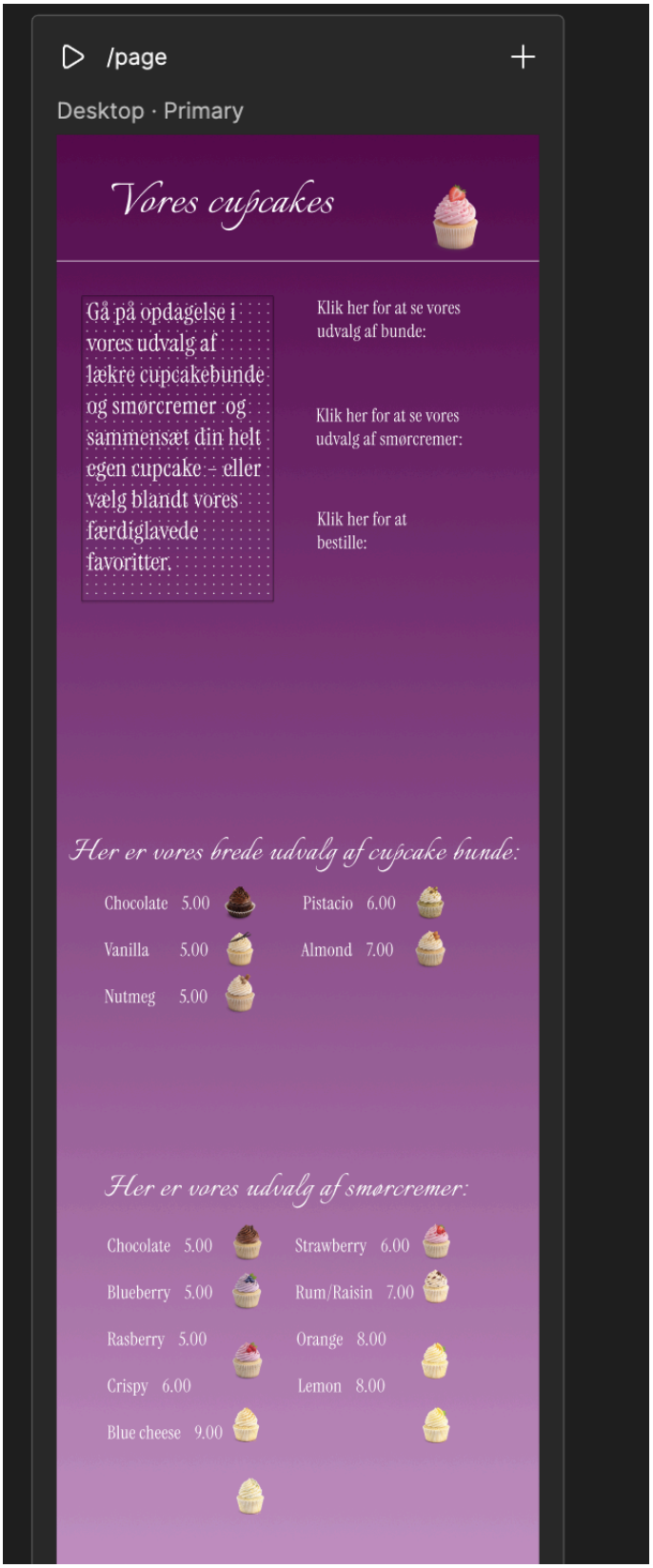


Bilag 4




Home
Desktop · Primary
log in
Olsker cupcakes - mums filibaba
Små cupcakes, store smil
Vores cupcakes
Om os
Kontakt

Bilag 5:


/page
Desktop · Primary
Vores cupcakes
Gå på opdagelse i vores udvalg af lækre cupcakebunde og smørcremer og sammensæt din helt egen cupcake - eller vælg blandt vores færdiglavede favoritter.
Klik her for at se vores udvalg af bunde:
Klik her for at se vores udvalg af smørcremer:
Klik her for at bestille:
Her er vores brede udvalg af cupcake bunde:
Chocolate 5.00
Vanilla 5.00
Nutmeg 5.00
Pistacio 6.00
Almond 7.00
Her er vores udvalg af smørcremer:
Chocolate 5.00
Blueberry 5.00
Rasberry 5.00
Crispy 6.00
Blue cheese 9.00
Strawberry 6.00
Rum/Raisin 7.00
Orange 8.00
Lemon 8.00

Bilag 6:

Desktop · Primary
Olsker cupcakes - mu
Små cupcakes, store
Vores cupcakes
Om os
Din kurv

bilag:7